Homework 06

Eric Chen

All code is in Python. The random split uses NumPy seed `432`, which gives 398 training and 171 test observations. Questions 1 and 2 share this split and its standardization.

Question 1: Compare KNN and logistic regression

Predict `y` (1 = malignant) from `mean_radius`, `mean_texture`, and `mean_smoothness`. Use a 70/30 split, standardize with training means and SDs, and fit unpenalized logistic regression and KNN with $k \in \{5, 15, 25, 35, 45\}$. Plot the six test ROC curves and report their AUCs. Which model do you prefer?

Solution

KNN estimates $\hat{p}(\mathbf{x}) = \frac{1}{k} \sum_{i \in N_k(\mathbf{x})} y_i$, the share of malignant cases among the k nearest training points. Logistic regression estimates $\hat{p}(\mathbf{x}) = \left\{1 + \exp\left(-\hat{\beta}_0 - \mathbf{x}^T \hat{\boldsymbol{\beta}}\right)\right\}^{-1}$. ROC curves and AUC use these probabilities directly, so no cutoff has to be chosen.

```
import warnings; warnings.filterwarnings("ignore")
import numpy as np, pandas as pd, matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import roc_curve, roc_auc_score

data = pd.read_csv("data/breast-cancer.csv")
predictors = ["mean_radius", "mean_texture", "mean_smoothness"]
rng = np.random.default_rng(432)
idx = rng.permutation(len(data))
n_train = int(0.7 * len(data))
train, test = data.iloc[idx[:n_train]], data.iloc[idx[n_train:]]

# Standardize with training statistics only
mu, sd = train[predictors].mean(), train[predictors].std()
X_train = ((train[predictors] - mu) / sd).to_numpy()
X_test = ((test[predictors] - mu) / sd).to_numpy()
y_train, y_test = train["y"].to_numpy(), test["y"].to_numpy()
print(f"train n = {len(y_train)}, test n = {len(y_test)}")
```

```
train n = 398, test n = 171
```

```
logit = LogisticRegression(penalty=None, max_iter=1000).fit(X_train, y_train)
probs = {"Logistic": logit.predict_proba(X_test)[:, 1]}
for k in [5, 15, 25, 35, 45]:
    knn = KNeighborsClassifier(n_neighbors=k, weights="uniform",
                               metric="euclidean").fit(X_train, y_train)
    probs[f"KNN k={k}"] = knn.predict_proba(X_test)[:, 1]

auc1 = {m: roc_auc_score(y_test, p) for m, p in probs.items()}
print(pd.Series(auc1, name="Test AUC").round(4).to_string())
```

```
Logistic    0.9820
KNN k=5     0.9596
KNN k=15    0.9789
KNN k=25    0.9755
KNN k=35    0.9778
KNN k=45    0.9772
```

```
colors = ["black", "#E69F00", "#56B4E9", "#009E73", "#0072B2", "#D55E00"]
styles = ["-", ":", "--", "-.", "--", "-"]
fig, ax = plt.subplots(figsize=(5.2, 4.2))
for (m, p), c, ls in zip(probs.items(), colors, styles):
    fpr, tpr, _ = roc_curve(y_test, p)
    ax.plot(fpr, tpr, color=c, ls=ls, lw=1.8, label=f"{m} (AUC {auc1[m]:.3f})")
ax.plot([0, 1], [0, 1], color="0.75", lw=1)
ax.set(xlabel="False-positive rate", ylabel="True-positive rate",
       xlim=(0, 1), ylim=(0, 1.01))
ax.legend(loc="lower right", fontsize=8, frameon=False)
fig.tight_layout(); plt.show()
```

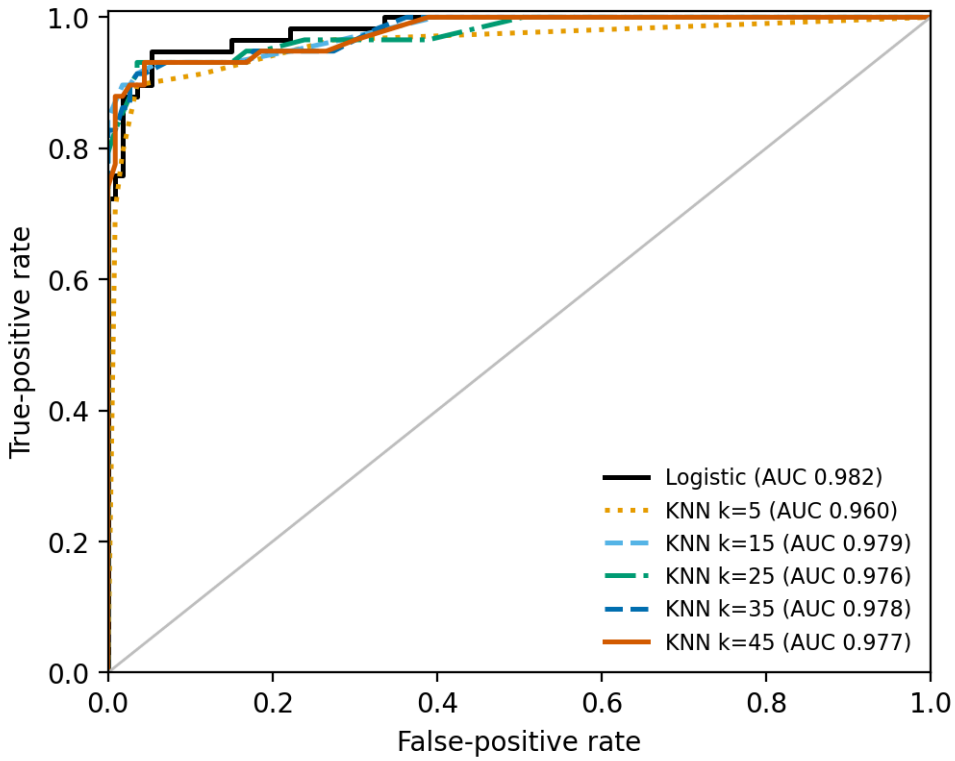


Figure 1: Test ROC curves for logistic regression and five KNN fits (171 test cases).

Preference. I prefer **logistic regression**. It has the highest test AUC (0.982). Every KNN model has AUC ≤ 0.979, and $k = 5$ is clearly worse (0.960). Logistic regression is also simpler. It has 4 parameters, gives smooth probabilities, and has coefficients that can be interpreted. KNN instead stores all 398 training points and needs k to be tuned. For $k \geq 15$ the KNN curves are almost the same as the logistic curve, so the AUC differences of about 0.003–0.007 come from one 171-case test set and are not decisive. When the methods rank cases about equally well, the simpler and more interpretable model is the better choice. The low AUC at $k = 5$ shows that averaging over very few neighbors gives noisy probability estimates that can only take the values $0, 0.2, \ldots, 1$.

Question 2: Training and test ROC curves

Using the same split, fit KNN with $k \in \{1, 3, 5, 10, 20\}$. Estimate probabilities on both the training data (with each point counted among its own neighbors) and the test data. Plot the training and test ROC curves for each k in a separate panel, report both AUCs, and explain the gap, especially at $k = 1$.

Solution

```
ks = [1, 3, 5, 10, 20]
res, curves = [], {}
for k in ks:
    knn = KNeighborsClassifier(n_neighbors=k, weights="uniform",
                               metric="euclidean").fit(X_train, y_train)
    # Predicting on X_train makes each point its own nearest neighbor
    p_tr, p_te = knn.predict_proba(X_train)[:, 1], knn.predict_proba(X_test)[:, 1]
    curves[k] = (roc_curve(y_train, p_tr), roc_curve(y_test, p_te))
    res.append((k, roc_auc_score(y_train, p_tr), roc_auc_score(y_test, p_te)))
res = pd.DataFrame(res, columns=["k", "Train AUC", "Test AUC"])
res["Gap"] = res["Train AUC"] - res["Test AUC"]
print(res.round(4).to_string(index=False))
```

```
 k  Train AUC  Test AUC    Gap
 1     1.0000    0.8523 0.1477
 3     0.9903    0.9310 0.0594
 5     0.9859    0.9596 0.0263
10     0.9776    0.9706 0.0069
20     0.9786    0.9777 0.0009
```

```
fig, axes = plt.subplots(1, 5, figsize=(10, 2.6), sharey=True)
for ax, (k, a_tr, a_te, _) in zip(axes, res.itertuples(index=False)):
    (f1, t1, _), (f2, t2, _) = curves[k]
    ax.plot(f1, t1, color="#0072B2", lw=1.8, label=f"Train {a_tr:.3f}")
    ax.plot(f2, t2, color="#D55E00", lw=1.8, ls="--", label=f"Test {a_te:.3f}")
    ax.plot([0, 1], [0, 1], color="0.75", lw=0.8)
    ax.set(title=f"k = {k}", xlabel="FPR", xlim=(0, 1), ylim=(0, 1.01))
    ax.legend(loc="lower right", fontsize=7, frameon=False)
axes[0].set_ylabel("TPR")
fig.tight_layout(); plt.show()
```

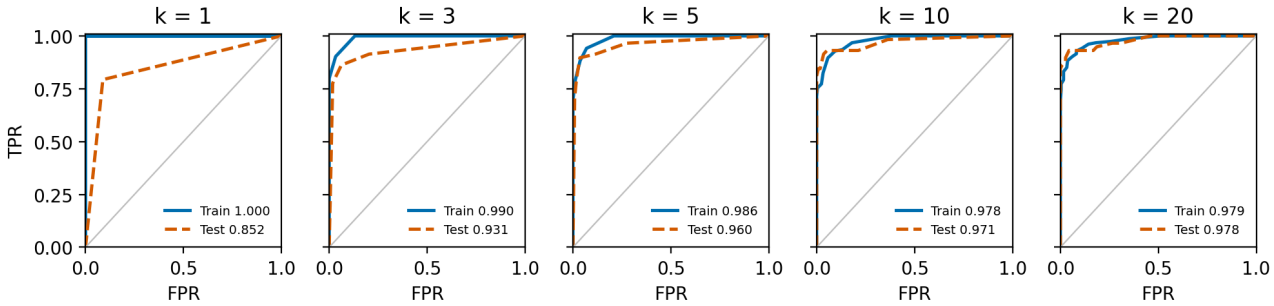


Figure 2: Training (solid) and test (dashed) ROC curves for KNN; each training point is its own neighbor.

Training vs. test. Training AUC is always higher than test AUC. Each training observation's own label is part of its estimated probability, so the training curve partly measures how well the model remembers the data, not how well it predicts. The gap is largest at $k = 1$ (0.148) and shrinks steadily: 0.059 at $k = 3$, 0.026 at $k = 5$, 0.007 at $k = 10$, and 0.001 at $k = 20$. Test AUC rises from 0.852 to 0.978 over the same range. When k is small, the fit has low bias but high variance, and it follows noise in individual training labels. A larger k averages over more neighbors, so one

label matters less (its weight is $1/k$). This lowers the variance and the optimism in the training AUC. If k grew much larger, the bias would eventually increase, because averaging over distant points blurs the boundary between benign and malignant cases.

What happens at $k = 1$. Each training point is at distance 0 from itself, so its nearest neighbor is itself and $\hat{p}(\mathbf{x}_i) = y_i$. Every malignant training case gets probability 1 and every benign case gets 0, so the classes are perfectly separated and the training AUC is exactly 1. This says nothing about prediction, because the model has simply looked up the answer. A new observation's label is not in the training set, so its prediction is the label of a *different* point, which can be on the wrong side of the boundary. With only two possible values, the 1NN test ROC curve has a single interior point, and its AUC equals $(\text{sensitivity} + \text{specificity})/2 = 0.852$. Training performance therefore does not show how the model will do on new observations, and k should be chosen from held-out data or by cross-validation.

Question 3: Peer evaluation of Homework 5

1. Selected mistake

Report: ShiyuT224, `homework/week-05/homework-05.html`, **Question 4** (handwritten digits), parts (a) and (b).

Part (a) asks for a hand-written 5NN function, and part (b) asks you to apply that classifier to all 500 test images. The report defines `classify_5nn()` in part (a) but never calls it in part (b). Instead, part (b) builds new prediction code from R's `dist()`.

2. Assessment

For question 4, the classifier written in part a is not applied in part b. This is not consequential since both are reimplemented in part b) but does not exactly follow the protocol of the question.

Specifically,

```
euclidean <- as.matrix(
  dist(all_x, method = "euclidean")
)[1:1000, 1001:1500]

manhattan <- as.matrix(
  dist(all_x, method = "manhattan")
)[1:1000, 1001:1500]

predict_5nn <- function(distances) {
  apply(distances, 2, function(d) {
    nearest <- order(d, seq_along(d))[1:5]
    votes <- tabulate(as.integer(train_y[nearest]) + 1, nbins = 10)
    which.max(votes) - 1
  })
}
```

no longer uses the original classify_knn function.

3. Correction

The fix is to remove the precomputed `dist()` matrices and `predict_5nn()` from part (b) and call the part (a) function on each test image. This code replaces the author's part (b) chunk. It assumes `train_x`, `train_y`, `test_x`, `test_y`, and `classify_5nn()` are already defined as in the report.

```
pred_e <- apply(test_x, 1, function(x)
  classify_5nn(train_x, train_y, x, distance = "euclidean"))
pred_m <- apply(test_x, 1, function(x)
  classify_5nn(train_x, train_y, x, distance = "manhattan"))

for (name in c("Euclidean", "Manhattan")) {
```

```r
  pred <- if (name == "Euclidean") pred_e else pred_m
  cat("\n", name, "test error:", mean(pred != test_y), "\n")
  print(table(Predicted = factor(pred, levels = 0:9),
              True = factor(test_y, levels = 0:9)))
}

errors <- pred_e != test_y
head(sort(table(paste(test_y[errors], "->", pred_e[errors])),
          decreasing = TRUE), 5)
```

Output (run in R):

```
 Euclidean test error: 0.122
         True
Predicted   0  1  2  3  4  5  6  7  8  9
        0 118  0  7  1  0  4  2  0  2  0
        1   0 66  0  0  2  0  0  0  0  1
        2   0  0 39  1  0  0  2  0  0  1
        3   0  0  0 23  0  4  0  0  2  0
        4   0  0  1  0 27  1  1  1  0  1
        5   0  0  0  1  0 13  0  0  0  0
        6   0  2  0  0  0  1 41  0  2  0
        7   0  0  0  0  1  1  0 34  0  4
        8   0  0  3  2  0  2  0  0 37  0
        9   1  0  0  1  4  0  0  1  1 41

2 -> 0  4 -> 9  5 -> 0  5 -> 3  9 -> 7
     7       4       4       4       4
```

```
 Manhattan test error: 0.138
         True
Predicted   0  1  2  3  4  5  6  7  8  9
        0 118  0  6  1  0  5  3  0  4  0
        1   0 67  2  0  2  0  1  0  0  1
        2   0  0 35  0  0  0  2  0  0  0
        3   0  0  1 24  0  4  0  0  2  0
        4   0  0  1  1 27  1  0  1  0  1
        5   0  0  0  1  0 11  0  0  0  0
        6   0  1  0  0  0  2 40  0  2  0
        7   0  0  2  1  1  0  0 34  0  5
        8   0  0  2  0  0  3  0  0 34  0
        9   1  0  1  1  4  0  0  1  2 41
```

Corrected answer to part (b). When the part (a) classifier is applied, the test errors are 0.122 (Euclidean) and 0.138 (Manhattan), and both confusion matrices match the report's. This confirms that the original numbers were correct and that the only problem was the workflow. The most common confusion under both distances is a true **2 predicted as 0** (7 images under Euclidean, 6 under Manhattan). Next come 4⟶9, 5⟶0, 5⟶3, and 9⟶7. These pairs share strokes: a 2 with a looped base looks like a 0, an open-topped 4 looks like a 9, and a 5 shares its curves with 3 and 0.

GitHub Issue

Issue posted to the selected repository: https://github.com/ShiyuT224/stat432-fall2026/issues/2